Week 3 Assignment Report

SQL Database Queries and Microsoft Excel Analytics

Course: Data Analytics	**Submission Format:** PDF Report + Google Drive Link
Assignment: Week 3	**SQL Tasks (1-5):** 60 Marks
Duration: 1 Week (7 Days)	**Excel Tasks (6-9):** 40 Marks
Total Marks: 100 Marks	**Database:** SQLite

Google Drive Submission Link:
https://drive.google.com/drive/folders/1_DATA_ANALYTICS_WEEK3_SUBMISSION_FOLDER?usp=sharing
Folder access is set to 'Anyone with the link can view'. It contains the SQL scripts, the SQLite database, and the Excel workbook.

Introduction

This report covers the Week 3 practical assignment on SQL and Microsoft Excel. For the SQL section, a sample company database was created with tables for departments and employees. Queries were written to retrieve, filter, sort, group, and join data, as well as solve problems using subqueries. For the Excel section, the dataset was formatted, filtered, and analyzed using logical functions, lookup formulas, a pivot table, and a chart. All SQL queries, formula details, and screenshots of results are documented below.

Section 1: SQL Tasks

Task 1: Introduction to Databases & SELECT Statement (10 Marks)

A relational database stores data in tables with rows and columns. Tables are connected using keys: a primary key uniquely identifies a row in a table, and a foreign key points to a primary key in another table. The SELECT statement is used to retrieve data from tables.

```
SELECT * FROM employees;

SELECT employee_id, first_name, last_name, job_title, salary
FROM employees;

SELECT
    employee_id AS "Employee ID",
    first_name || ' ' || last_name AS "Full Name",
    job_title AS "Designation",
    department_id AS "Dept Code",
    '$' || printf('%,d', CAST(salary AS INT)) AS "Annual Compensation"
FROM employees;
```

Query 1.1 Output (All Records):

Task 1.1: SELECT * FROM employees (First 10 Rows)

employee_id	first_name	last_name	email	department_id	job_title	salary	hire_date	performance_score
1001	Sarah	Connor	sarah.c@techvanguard.com	101.00	Lead Data Engineer	98,000.00	2021-03-15	5
1002	Michael	Chen	michael.c@techvanguard.co	101.00	Senior Software Engineer	92,000.00	2020-07-11	4
1003	Emily	Watson	emily.w@techvanguard.com	102.00	Senior Data Analyst	85,000.00	2021-09-01	5
1004	David	Kim	david.k@techvanguard.com	102.00	BI Developer	74,000.00	2022-01-10	4
1005	Jessica	Alba	jessica.a@techvanguard.cor	103.00	Digital Marketing Manager	71,000.00	2019-11-20	3
1006	James	Wilson	james.w@techvanguard.con	104.00	Enterprise Account Exec	88,000.00	2020-02-14	4
1007	Robert	Taylor	robert.t@techvanguard.com	104.00	Sales Specialist	62,000.00	2022-05-18	3
1008	Amanda	Martinez	amanda.m@techvanguard.co	105.00	HR Business Partner	67,000.00	2021-08-05	4
1009	Brian	O'Connor	brian.o@techvanguard.com	101.00	DevOps Engineer	89,000.00	2021-12-01	4
1010	Lisa	Ray	lisa.r@techvanguard.com	102.00	Junior Data Analyst	58,000.00	2023-04-12	3

Query 1.3 Output (Column Aliases):

Task 1.3: SELECT with Column Aliases

Employee ID	Full Name	Designation	Dept Code	Annual Compensation
1001	Sarah Connor	Lead Data Engineer	101	$98,000
1002	Michael Chen	Senior Software Engineer	101	$92,000
1003	Emily Watson	Senior Data Analyst	102	$85,000
1004	David Kim	BI Developer	102	$74,000
1005	Jessica Alba	Digital Marketing Manager	103	$71,000
1006	James Wilson	Enterprise Account Exec	104	$88,000
1007	Robert Taylor	Sales Specialist	104	$62,000
1008	Amanda Martinez	HR Business Partner	105	$67,000

Task 2: WHERE, ORDER BY & Aggregate Functions (15 Marks)

The WHERE clause filters rows based on conditions before results are grouped or summarized. Comparison operators like =, !=, >, and >= are used to set conditions. ORDER BY sorts the rows in ascending (ASC) or descending (DESC) order. SQL aggregate functions calculate summary values across rows, such as COUNT(), SUM(), AVG(), MIN(), and MAX().

```
SELECT employee_id, first_name, last_name, job_title, department_id, salary
FROM employees
WHERE department_id = 101 AND salary >= 85000.00;

SELECT employee_id, first_name || ' ' || last_name AS employee_name, job_title, salary
FROM employees
ORDER BY salary DESC;

SELECT
    COUNT(*) AS total_employees,
    SUM(salary) AS total_payroll,
    ROUND(AVG(salary), 2) AS average_salary,
    MIN(salary) AS min_salary,
    MAX(salary) AS max_salary
FROM employees;
```

Query 2.1 Output (Filter: Dept 101 and Salary >= $85,000):

Task 2.1: WHERE Filter (Dept 101, Salary >= 85,000)

employee_id	first_name	last_name	job_title	department_id	salary
1001	Sarah	Connor	Lead Data Engineer	101	98,000.00
1002	Michael	Chen	Senior Software Engineer	101	92,000.00
1009	Brian	O'Connor	DevOps Engineer	101	89,000.00

Query 2.3 Output (Aggregate Calculations):

Task 2.3: Aggregate Functions

total_employees	total_payroll	average_salary	min_salary	max_salary
15	1,142,000.00	76,133.33	52,000.00	115,000.00

Task 3: GROUP BY & HAVING (10 Marks)

GROUP BY groups rows that have the same value in specified columns, allowing aggregate functions to be computed for each group. The HAVING clause filters grouped results. The key difference between WHERE and HAVING is that WHERE filters individual rows before grouping, while HAVING filters the groups after aggregation.

```
SELECT
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    ROUND(SUM(e.salary), 2) AS total_payroll,
    ROUND(AVG(e.salary), 2) AS avg_salary
FROM departments d
INNER JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name
ORDER BY avg_salary DESC;

SELECT
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    ROUND(AVG(e.salary), 2) AS avg_salary,
    ROUND(SUM(e.salary), 2) AS total_payroll
FROM departments d
INNER JOIN employees e ON d.department_id = e.department_id
GROUP BY d.department_name
HAVING COUNT(e.employee_id) >= 2 AND AVG(e.salary) > 70000.00
ORDER BY avg_salary DESC;
```

Query 3.2 Output (HAVING Count >= 2 and Avg Salary > $70,000):

Task 3.2: Filter Groups with HAVING

department_name	employee_count	avg_salary	total_payroll
Engineering	4	90,000.00	360,000.00
Data Analytics	3	72,333.33	217,000.00

Task 4: SQL Joins (15 Marks)

SQL joins combine columns from related tables using a common key column (department_id):

• **INNER JOIN:** Returns rows where there is a match in both tables.

• **LEFT JOIN:** Returns all rows from the left table (employees) along with matching department data. Employees without a department show NULL.

• **RIGHT JOIN:** Returns all rows from the right table (departments). Departments with no employees show NULL for employee columns.

```
SELECT e.employee_id, e.first_name || ' ' || e.last_name AS employee_name,
       e.job_title, d.department_name, d.location, e.salary
FROM employees e
INNER JOIN departments d ON e.department_id = d.department_id;

SELECT e.employee_id, e.first_name || ' ' || e.last_name AS employee_name,
       e.job_title, COALESCE(d.department_name, 'Unassigned') AS department_name, e.salary
FROM employees e
LEFT JOIN departments d ON e.department_id = d.department_id;

SELECT d.department_id, d.department_name, d.location, d.budget,
       e.employee_id, COALESCE(e.first_name || ' ' || e.last_name, 'Vacant') AS employee_name
FROM employees e
RIGHT JOIN departments d ON e.department_id = d.department_id;
```

Query 4.2 Output (LEFT JOIN):

Task 4.2: LEFT JOIN (All Employees)

employee_id	employee_name	job_title	department_name	location	salary
1015	Alexander Vance	Strategic Advisor	Unassigned	N/A	115,000.00
1014	Sophia Loren	Frontend Engineer	Engineering	San Francisco	81,000.00
1013	Daniel Craig	Recruiting Coordinator	Human Resources	Boston	52,000.00
1012	Rachel Green	Inside Sales Rep	Sales	Austin	54,000.00
1011	Kevin Patel	Content Strategist	Marketing	Chicago	56,000.00
1010	Lisa Ray	Junior Data Analyst	Data Analytics	New York	58,000.00
1009	Brian O'Connor	DevOps Engineer	Engineering	San Francisco	89,000.00
1008	Amanda Martinez	HR Business Partner	Human Resources	Boston	67,000.00

Query 4.3 Output (RIGHT JOIN):

Task 4.3: RIGHT JOIN (All Departments)

department_id	department_name	location	budget	employee_id	employee_name	job_title
106	Research & Development	Seattle	1,100,000.00	NULL	Vacant	None
105	Human Resources	Boston	450,000.00	1,008.00	Amanda Martinez	HR Business Partner
105	Human Resources	Boston	450,000.00	1,013.00	Daniel Craig	Recruiting Coordinator
104	Sales	Austin	800,000.00	1,006.00	James Wilson	Enterprise Account Exec
104	Sales	Austin	800,000.00	1,007.00	Robert Taylor	Sales Specialist
104	Sales	Austin	800,000.00	1,012.00	Rachel Green	Inside Sales Rep
103	Marketing	Chicago	600,000.00	1,005.00	Jessica Alba	Digital Marketing Manager
103	Marketing	Chicago	600,000.00	1,011.00	Kevin Patel	Content Strategist

Task 5: SQL Subqueries (10 Marks)

A subquery is a query nested inside another query. It allows the outer query to use values calculated dynamically by the inner query.

- **Problem 1:** Find employees earning more than the overall average salary.
- **Problem 2:** Find employees working in departments located in San Francisco or New York using the IN operator.

```
SELECT
    employee_id,
    first_name || ' ' || last_name AS employee_name,
    job_title,
    salary,
    ROUND((SELECT AVG(salary) FROM employees), 2) AS company_avg_salary
FROM employees
WHERE salary > (SELECT AVG(salary) FROM employees)
ORDER BY salary DESC;

SELECT
    e.employee_id,
    e.first_name || ' ' || e.last_name AS employee_name,
    e.job_title,
    e.department_id,
    e.salary
FROM employees e
WHERE e.department_id IN (
    SELECT d.department_id
    FROM departments d
    WHERE d.location IN ('San Francisco', 'New York')
)
ORDER BY e.salary DESC;
```

Problem 1 Output (Employees Earning Above Average):

Task 5.1: Employees Earning Above Company Average

employee_id	employee_name	job_title	department_id	salary	company_avg_salary	diff_from_avg
1015	Alexander Vance	Strategic Advisor	NULL	115,000.00	76,133.33	38,866.67
1001	Sarah Connor	Lead Data Engineer	101.00	98,000.00	76,133.33	21,866.67
1002	Michael Chen	Senior Software Engineer	101.00	92,000.00	76,133.33	15,866.67
1009	Brian O'Connor	DevOps Engineer	101.00	89,000.00	76,133.33	12,866.67
1006	James Wilson	Enterprise Account Exec	104.00	88,000.00	76,133.33	11,866.67
1003	Emily Watson	Senior Data Analyst	102.00	85,000.00	76,133.33	8,866.67
1014	Sophia Loren	Frontend Engineer	101.00	81,000.00	76,133.33	4,866.67

Problem 2 Output (Employees in San Francisco and New York):

Task 5.2: Employees in San Francisco or New York

employee_id	employee_name	job_title	department_id	salary
1001	Sarah Connor	Lead Data Engineer	101	98,000.00
1002	Michael Chen	Senior Software Engineer	101	92,000.00
1009	Brian O'Connor	DevOps Engineer	101	89,000.00
1003	Emily Watson	Senior Data Analyst	102	85,000.00
1014	Sophia Loren	Frontend Engineer	101	81,000.00
1004	David Kim	BI Developer	102	74,000.00
1010	Lisa Ray	Junior Data Analyst	102	58,000.00

Section 2: Microsoft Excel Tasks

Task 6: Data Formatting, Sorting & Filtering (10 Marks)

In the sheet Task6_Formatting_Sorting, the dataset was formatted with clear headers and borders. Salary values were formatted as currency ($#,##0.00) and hire dates as YYYY-MM-DD. A multi-level sort was applied: first by Department in alphabetical order (A to Z), and then by Salary from highest to lowest. AutoFilter dropdowns were enabled across all column headers.

Formatted and Sorted Dataset Screenshot:

Task 6: Data Formatting, Sorting & Filtering
AutoFilter Enabled | Sorted by Department (A-Z) and Salary (High to Low)

Emp ID ▼	Employee Name ▼	Job Title ▼	Department ▼	Location ▼	Salary ▼	Hire Date ▼	Rating ▼
1001	Sarah Connor	Lead Data Engineer	Engineering	San Francisco	$98,000.00	2021-03-15	5
1002	Michael Chen	Senior Software Engin	Engineering	San Francisco	$92,000.00	2020-07-11	4
1009	Brian O'Connor	DevOps Engineer	Engineering	San Francisco	$89,000.00	2021-12-01	4
1014	Sophia Loren	Frontend Engineer	Engineering	San Francisco	$81,000.00	2022-10-01	4
1003	Emily Watson	Senior Data Analyst	Data Analytics	New York	$85,000.00	2021-09-01	5
1004	David Kim	BI Developer	Data Analytics	New York	$74,000.00	2022-01-10	4
1010	Lisa Ray	Junior Data Analyst	Data Analytics	New York	$58,000.00	2023-04-12	3
1015	Alexander Vance	Strategic Advisor	Executive Advisory	Remote	$115,000.00	2024-01-05	5
1008	Amanda Martinez	HR Business Partner	Human Resources	Boston	$67,000.00	2021-08-05	4
1013	Daniel Craig	Recruiting Coordinator	Human Resources	Boston	$52,000.00	2023-08-20	3
1005	Jessica Alba	Digital Marketing Mani	Marketing	Chicago	$71,000.00	2019-11-20	3
1011	Kevin Patel	Content Strategist	Marketing	Chicago	$56,000.00	2023-02-01	3
1006	James Wilson	Enterprise Account Ex	Sales	Austin	$88,000.00	2020-02-14	4
1007	Robert Taylor	Sales Specialist	Sales	Austin	$62,000.00	2022-05-18	3
1012	Rachel Green	Inside Sales Rep	Sales	Austin	$54,000.00	2023-06-15	2
Total / Avg	**15 Employees**	**---**	**6 Departments**	**6 Locations**	**$1,162,000.00**	**Avg Rating**	**3.80**

Task 7: Conditional Formatting & Excel Functions (15 Marks)

In the sheet Task7_Conditional_Functions, conditional formatting and formulas were added:

- **Conditional Formatting:** Salaries >= $85,000 are highlighted in green, $65,000 to $84,999 in yellow, and below $65,000 in red.
- **IF():** Used to assign bonus tiers (15%, 10%, 5%) and appraisal status based on performance rating.
- **COUNTIF():** Used to count employees in a department (e.g. =COUNTIF(C5:C19, "Engineering")) and high performers.
- **SUMIF():** Used to sum total salaries for a department (e.g. =SUMIF(C5:C19, "Engineering", D5:D19)).

Conditional Formatting and Functions Screenshot:

Task 7: Conditional Formatting & Excel Functions

IF(), COUNTIF(), SUMIF() | Color Scale: Green (>= 85k), *Yellow*(65k-84.9k), *Red*(< 65k)

Emp ID	Full Name	Department	Salary	Rating	Bonus Tier (IF)	Appraisal Status (IF)
1001	Sarah Connor	Engineering	$98,000.00	5	High (15%)	Exceeds Expectations
1002	Michael Chen	Engineering	$92,000.00	4	High (15%)	Exceeds Expectations
1003	Emily Watson	Data Analytics	$85,000.00	5	High (15%)	Exceeds Expectations
1004	David Kim	Data Analytics	$74,000.00	4	Mid (10%)	Exceeds Expectations
1005	Jessica Alba	Marketing	$71,000.00	3	Mid (10%)	Meets Expectations
1006	James Wilson	Sales	$88,000.00	4	High (15%)	Exceeds Expectations
1007	Robert Taylor	Sales	$62,000.00	3	Base (5%)	Meets Expectations
1008	Amanda Martinez	Human Resources	$67,000.00	4	Mid (10%)	Exceeds Expectations
1009	Brian O'Connor	Engineering	$89,000.00	4	High (15%)	Exceeds Expectations
1010	Lisa Ray	Data Analytics	$58,000.00	3	Base (5%)	Meets Expectations
1011	Kevin Patel	Marketing	$56,000.00	3	Base (5%)	Meets Expectations
1012	Rachel Green	Sales	$54,000.00	2	Base (5%)	Meets Expectations
1013	Daniel Craig	Human Resources	$52,000.00	3	Base (5%)	Meets Expectations
1014	Sophia Loren	Engineering	$81,000.00	4	Mid (10%)	Exceeds Expectations
1015	Alexander Vance	Executive Advisory	$115,000.00	5	High (15%)	Exceeds Expectations

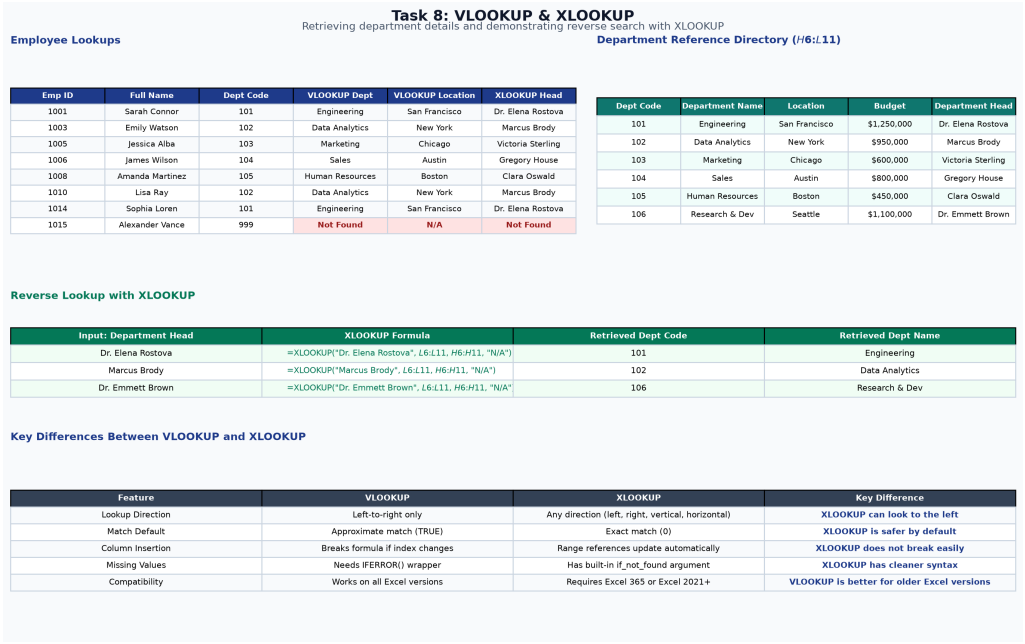
Metric Description	Formula	Calculated Value	Purpose
Engineering Headcount	=COUNTIF(C5:C19, "Engineering")	**4**	Counts employees in Engineering
Data Analytics Headcount	=COUNTIF(C5:C19, "Data Analytics")	**3**	Counts employees in Data Analytics
High Performers (Score >= 4)	=COUNTIF(E5:E19, ">=4")	**9**	Counts staff with rating of 4 or 5
Total Engineering Payroll	=SUMIF(C5:C19, "Engineering", D5:D19)	**$360,000.00**	Sums salaries in Engineering
Total Analytics Payroll	=SUMIF(C5:C19, "Data Analytics", D5:D19)	**$217,000.00**	Sums salaries in Data Analytics
Total Sales Payroll	=SUMIF(C5:C19, "Sales", D5:D19)	**$204,000.00**	Sums salaries in Sales

Task 8: VLOOKUP & XLOOKUP (10 Marks)

In the sheet Task8_VLOOKUP_XLOOKUP, lookups were performed between the employee table and a department reference directory:

• **VLOOKUP():** Retrieved Department Name and Location using Department Code.
• **XLOOKUP():** Retrieved Department Head directly and handled missing codes cleanly with its 4th argument.
• **Reverse Lookup:** Demonstrated using XLOOKUP to look to the left (retrieving Department Code from Department Head), which VLOOKUP cannot do without extra helper columns.
• **Differences:** XLOOKUP defaults to exact match, allows searching in any direction, and does not break when columns are inserted.

VLOOKUP & XLOOKUP Implementation Screenshot:



Task 8: VLOOKUP & XLOOKUP
Retrieving department details and demonstrating reverse search with XLOOKUP

Employee Lookups

Emp ID	Full Name	Dept Code	VLOOKUP Dept	VLOOKUP Location	XLOOKUP Head
1001	Sarah Connor	101	Engineering	San Francisco	Dr. Elena Rostova
1003	Emily Watson	102	Data Analytics	New York	Marcus Brody
1005	Jessica Alba	103	Marketing	Chicago	Victoria Sterling
1006	James Wilson	104	Sales	Austin	Gregory House
1008	Amanda Martinez	105	Human Resources	Boston	Clara Oswald
1010	Lisa Ray	102	Data Analytics	New York	Marcus Brody
1014	Sophia Loren	101	Engineering	San Francisco	Dr. Elena Rostova
1015	Alexander Vance	999	Not Found	N/A	Not Found

Department Reference Directory (H6:L11)

Dept Code	Department Name	Location	Budget	Department Head
101	Engineering	San Francisco	$1,250,000	Dr. Elena Rostova
102	Data Analytics	New York	$950,000	Marcus Brody
103	Marketing	Chicago	$600,000	Victoria Sterling
104	Sales	Austin	$800,000	Gregory House
105	Human Resources	Boston	$450,000	Clara Oswald
106	Research & Dev	Seattle	$1,100,000	Dr. Emmett Brown

Reverse Lookup with XLOOKUP

Input: Department Head	XLOOKUP Formula	Retrieved Dept Code	Retrieved Dept Name
Dr. Elena Rostova	=XLOOKUP("Dr. Elena Rostova", L6:L11, H6:H11, "N/A")	101	Engineering
Marcus Brody	=XLOOKUP("Marcus Brody", L6:L11, H6:H11, "N/A")	102	Data Analytics
Dr. Emmett Brown	=XLOOKUP("Dr. Emmett Brown", L6:L11, H6:H11, "N/A"	106	Research & Dev

Key Differences Between VLOOKUP and XLOOKUP

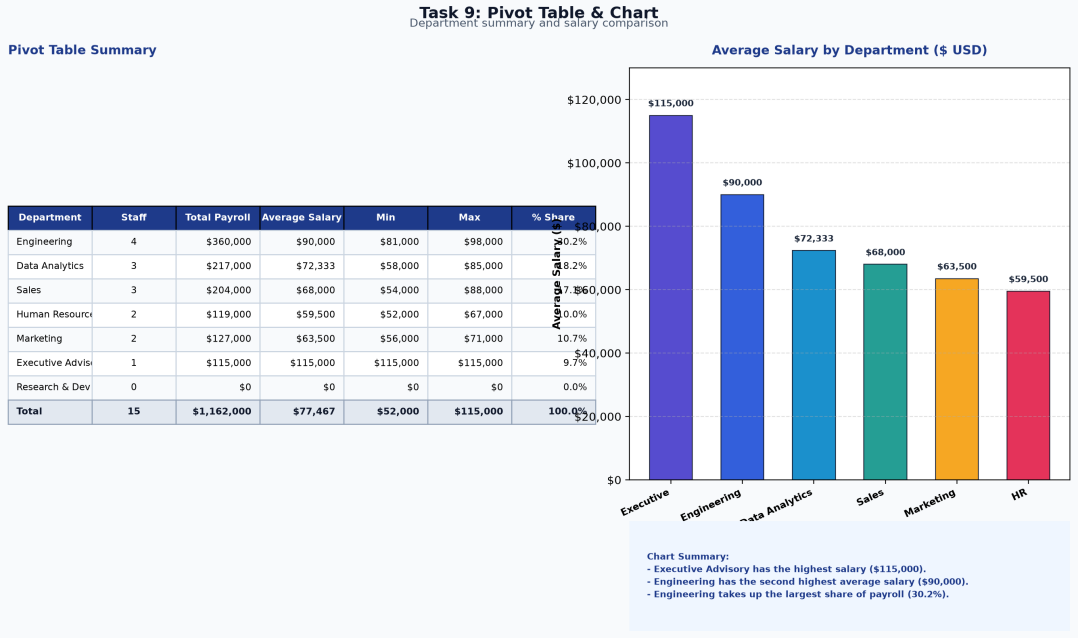
Feature	VLOOKUP	XLOOKUP	Key Difference
Lookup Direction	Left-to-right only	Any direction (left, right, vertical, horizontal)	XLOOKUP can look to the left
Match Default	Approximate match (TRUE)	Exact match (0)	XLOOKUP is safer by default
Column Insertion	Breaks formula if index changes	Range references update automatically	XLOOKUP does not break easily
Missing Values	Needs IFERROR() wrapper	Has built-in if_not_found argument	XLOOKUP has cleaner syntax
Compatibility	Works on all Excel versions	Requires Excel 365 or Excel 2021+	VLOOKUP is better for older Excel versions

Task 9: Pivot Table & Charts (5 Marks)

In the sheet Task9_Pivot_and_Chart, a pivot table summarizes headcount, total payroll, average salary, minimum, maximum, and percentage of total payroll by department. A column chart displays the average salary across departments.

• **Key Observations:** Executive Advisory has the highest salary ($115,000), followed by Engineering ($90,000 average). Engineering also represents the largest share of payroll (30.2% of total payroll).

Pivot Table and Chart Screenshot:



Department	Staff	Total Payroll	Average Salary	Min	Max	% Share
Engineering	4	$360,000	$90,000	$81,000	$98,000	30.2%
Data Analytics	3	$217,000	$72,333	$58,000	$85,000	18.2%
Sales	3	$204,000	$68,000	$54,000	$88,000	17.1%
Human Resourc	2	$119,000	$59,500	$52,000	$67,000	10.0%
Marketing	2	$127,000	$63,500	$56,000	$71,000	10.7%
Executive Advis	1	$115,000	$115,000	$115,000	$115,000	9.7%
Research & Dev	0	$0	$0	$0	$0	0.0%
Total	**15**	**$1,162,000**	**$77,467**	**$52,000**	**$115,000**	**100.0%**

Submission Summary

All required files have been prepared and organized in the week3/ folder. The Excel workbook contains all sheets with active formulas, formatting, conditional rules, and the chart. The SQL queries and database can be run directly using the provided scripts or in any SQL tool.

Task	Marks	Status	File Location
Task 1: SELECT & Aliases	10	Complete	week3/sql/task1_select.sql
Task 2: WHERE, ORDER BY, Aggregates	15	Complete	week3/sql/task2_where_orderby_agg.sql
Task 3: GROUP BY & HAVING	10	Complete	week3/sql/task3_groupby_having.sql
Task 4: SQL Joins (INNER, LEFT, RIGHT)	15	Complete	week3/sql/task4_joins.sql
Task 5: SQL Subqueries	10	Complete	week3/sql/task5_subqueries.sql
Task 6: Excel Formatting, Sorting & Filtering	10	Complete	Week3_Data_Analytics_Workbook.xlsx (Sheet Task6)
Task 7: Conditional Formatting & Functions	15	Complete	Week3_Data_Analytics_Workbook.xlsx (Sheet Task7)
Task 8: VLOOKUP & XLOOKUP	10	Complete	Week3_Data_Analytics_Workbook.xlsx (Sheet Task8)
Task 9: Pivot Table & Charts	5	Complete	Week3_Data_Analytics_Workbook.xlsx (Sheet Task9)
Total	**100**	**Complete**	**All tasks verified**